AutoML, Autoresearch, MLOps +@

DOE는 Research Agent의 Harness가 될 수 있는가?

배경 정리 + harness 제안

1. `Autoresearch`라는 말은 어디서 왔나?

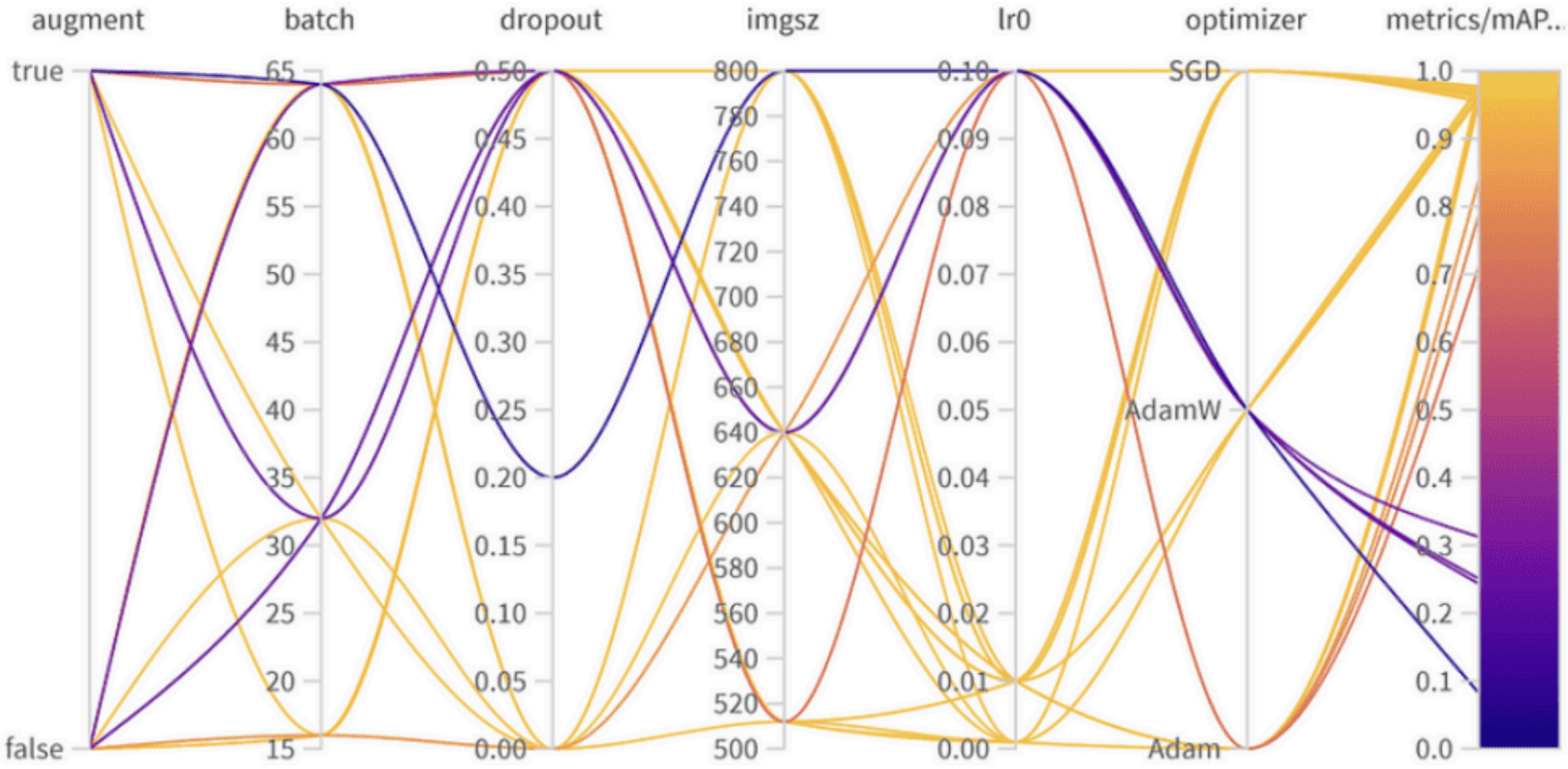
- 2026년 karpathy/autoresearch가 작은 but real training setup 위의 autonomous research loop를 대중적으로 제시했다.
- 핵심 아이디어는 agent가 `read → edit → run → keep-or-revert`를 반복하며 밤새 실험을 누적하는 것이다.
- 이후 SakanaAI/AI-Scientist, microsoft/RD-Agent, GPT Researcher 같은 계열로 빠르게 분화했다.
- 즉 `Autoresearch`는 단일 repo 이름을 넘어서 research automation의 한 계열 이름이 되고 있다.

2. AutoML이란 무엇인가?

- AutoML은 모델 개발의 탐색 일부를 자동화한다.
- 대표 대상은 `model selection`, `hyperparameter tuning`, `pipeline search`다.
- 핵심은 비교적 주어진 `search space` 안 최적화다.

```
Data → Search over models/pipelines → Evaluation → Best configuration
```

3. AutoML은 이런 그림으로 설명되는 경우가 많다



Source: visionplatform.ai

4. Autoresearch란 무엇인가?

- Autoresearch는 연구 workflow 일부를 agent가 직접 수행하는 형태다.
- 대상은 `config`가 아니라 `가설`, `code`, `pipeline`, `experiment plan`까지 넓어진다.
- 목표도 단일 최고 설정보다 더 나은 실험 program을 찾는 데 가깝다.

`Question → Read → Edit → Run → Analyze → Next experiment`

5. Autoresearch agent는 어떻게 일하는가?

단계	agent가 하는 일
context build	논문, 코드, 이전 결과를 읽는다
planning	다음 실험과 baseline을 고른다
execution	코드를 바꾸고 실행한다
interpretation	결과를 해석하고 다음 실험으로 넘긴다

- 즉 Autoresearch는 한 번의 search보다 `iterative research loop`에 가깝다.

6. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, architecture	hypothesis, code, module, pipeline, experiment program
평가	explicit objective	objective + reasoning + iteration
주요 위험	비효율적 탐색	metric hacking, incoherent search
필요한 인프라	experiment infra	experiment + memory + harness

7. 현재 사용례

작업	예시
문헌 조사 / deep research	GPT Researcher
코드 수정 + 실험 반복	karpathy/autoresearch, RD-Agent
end-to-end 연구 자동화	AI-Scientist
benchmark / evaluation	MLE-bench, MLAgentBench, MLR-Bench

8. 발전 방향

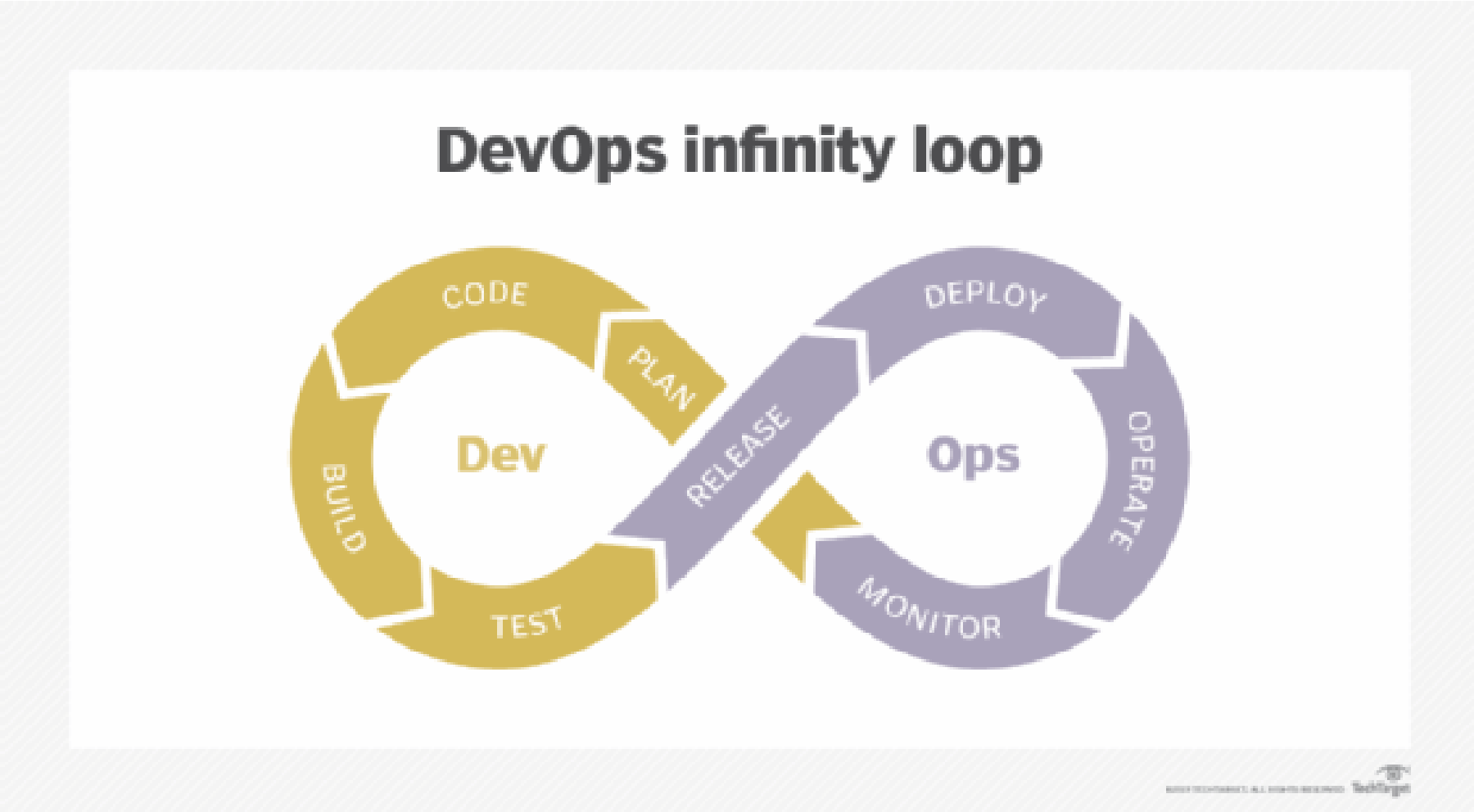
- 후속 ecosystem과 hardware fork가 빠르게 늘고 있다. awesome-autoresearch
- `idea → experiment → paper` 형 end-to-end stack이 두터워지고 있다. Awesome Auto Research Tools
- benchmark가 점점 기준점이 되고 있다. MLE-bench, MLAgentBench, MLR-Bench
- `memory`, `skills`, `plugin` 으로 연구 loop를 모듈화하는 흐름이 보인다. GPT Researcher, awesome-autoresearch

9. 왜 MLOps가 공통 기반이 되는가?

- AutoML도 Autoresearch도 결국 `많은 run`을 비교하고 누적하는 문제다.
- run 수가 커지면 `tracking`, `lineage`, `orchestration` 없이는 자동화가 지식으로 남지 않는다.
- agent가 코드를 바꾸기 시작하면 `artifact`, `promotion`, `monitoring`, `cost control`이 더 중요해진다.

MLOps가 없으면	생기는 문제
tracking 없음	최고 run를 잃는다
lineage 없음	왜 좋아졌는지 설명 못 한다
orchestration 없음	반복 loop가 수작업으로 돌아간다
monitoring 없음	배포 후 feedback이 끊긴다

10. MLOps는 결국 loop를 운영하는 discipline이다



MLOps는 이 반복 구조를 `data`, `model`, `artifact`, `deployment` 까지 확장한다.

loop 시각화: TechTarget, 2026-04-07 확인

11. 실무에서는 tracking UI가 지식 저장소가 된다

12. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

13. Prelim 정리

배경은 여기까지다.

이제 질문은

`Research Agent에 어떤 harness가 필요한가?`

14. Harness gap

Coding Agent 쪽에서 이미 흔한 것	Research Agent 쪽에서 아직 약한 것
`TDD`	hypothesis discipline
`CI`	factor definition
regression test	robustness / replication
deploy guardrail	experiment promotion rule

- 그래서 높은 점수만으로는 좋은 연구 탐색인지 말하기 어렵다.

15. DOE를 Harness 후보로 보기

DOE primitive	Agent loop에서의 역할
screening	중요한 factor를 초기에 좁힘
factorial thinking	interaction을 구조적으로 확인
sequential refinement	유망 구간을 단계적으로 세밀화
robustness / confirmation	우연한 win과 재현 가능한 win을 분리

- 즉 DOE는 실험 순서와 비교 규칙을 준다.

16. Agent loop에 무엇이 달라지는가?

단계	Vanilla Agent	DOE-guided Agent
초기	idea → edit → run	factor 정의 → screening
중반	gain 추적	interaction check
후반	best-so-far 갱신	refinement
승격	high score 채택	robustness 후 promote

17. 실험 설정

Variant	설명	비교 목적
Vanilla Agent	unconstrained iterative experiments	기본 baseline
DOE Agent	DOE 기반 planning과 staging	harness 효과 측정
DOE + X Agent	DOE + 추가 전략 모듈	harness와 추가 전략의 결합 효과

- 비교 축: `score`, `cost`, `robustness`, `portfolio behavior`

18. 결과 요약

실험 후 채움

Variant	Best score	Avg improvement	Runs	Cost	Robustness pass
Vanilla	TBD	TBD	TBD	TBD	TBD
DOE	TBD	TBD	TBD	TBD	TBD
DOE + X	TBD	TBD	TBD	TBD	TBD

19. 성능 향상 추이

- `실험 후 채움`
- `experiment budget` 또는 `run count` 대비 `best-so-far metric`
- 수렴 속도, plateau 형태, sample efficiency 비교

`x-axis = budget / run count`

`y-axis = best-so-far metric`

20. 실험 포트폴리오 또는 실패 구조

- `실험 후 채움`
- `Tic / Tac / To` 비율
- 또는 실패 분류
- 또는 `information gain vs. performance gain`

21. 왜 DOE-guided Agent는 다를 수 있는가?

- `search space`가 더 구조화된다.
- 중복되거나 정보량이 낮은 실험이 줄어든다.
- 상호작용을 우연히가 아니라 의도적으로 본다.
- robustness 단계가 분리된다.

22. 대학원 연구 루프는 더 넓다

- 문헌 조사
- 가설 수립
- 실험 설계
- 구현 및 실행
- 분석과 토론
- 다음 실험 선택

23. DOE가 주로 커버하는 구간

구간	DOE의 역할
강함	실험 설계, 비교, refinement, 다음 실험 선택
부분적	구현 및 실행, 결과 분석
약함	문헌 조사, 가설 생성

- 즉 DOE는 `research loop` 전체보다 experimentation layer를 강하게 규율한다.

24. 한계

- `literature review`나 `hypothesis generation` 자체를 대체하진 못한다.
- open-ended research space는 factorization이 어렵다.
- large-scale training에서는 replication 비용이 크다.
- evaluation harness가 약하면 DOE도 흔들린다.

25. 결론

- AutoML에서 Autoresearch로 갈수록 `search space` 는 넓어진다.
- search space가 넓어질수록 더 강한 `harness` 가 필요하다.
- DOE는 Research Agent의 experimentation harness로 매우 유력한 후보다.

> 다음 단계는 더 좋은 agent 자체보다 더 좋은 harness이다.

26. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
visuals / MLOps	visionplatform.ai YOLOv8 HPO, TechTarget MLOps lifecycle article, MLflow Tracking docs, Azure MLOps overview